INDIAN INSTITUTE OF TECHNOLOGY MADRAS

सिद्धिर्भवति कर्मजा

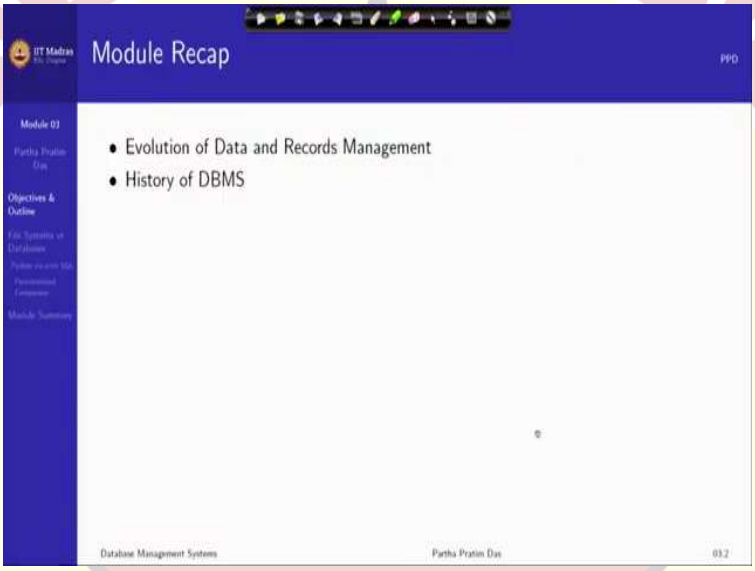
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Why DBMS?/2

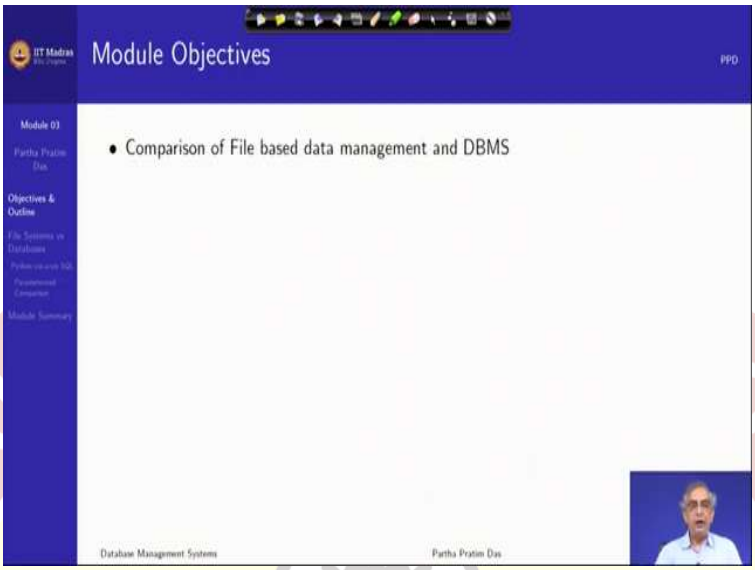
Welcome to the module 3 of database management systems course in the IIT Madras BSc online program. In the last module, we mentioned that we are going to discuss about issues relating to why DBMS over the two modules.

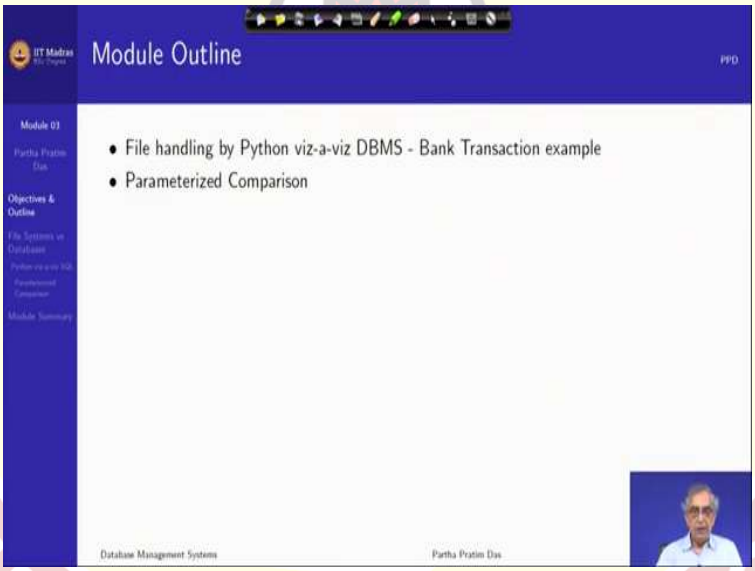
(Refer Slide Time: 0:44)



So, in the last module, we talked primarily about the evolution of data and record management, from physical systems to different generations of electronic systems and talked very briefly about the history of DBMS.

(Refer Slide Time: 1:02)



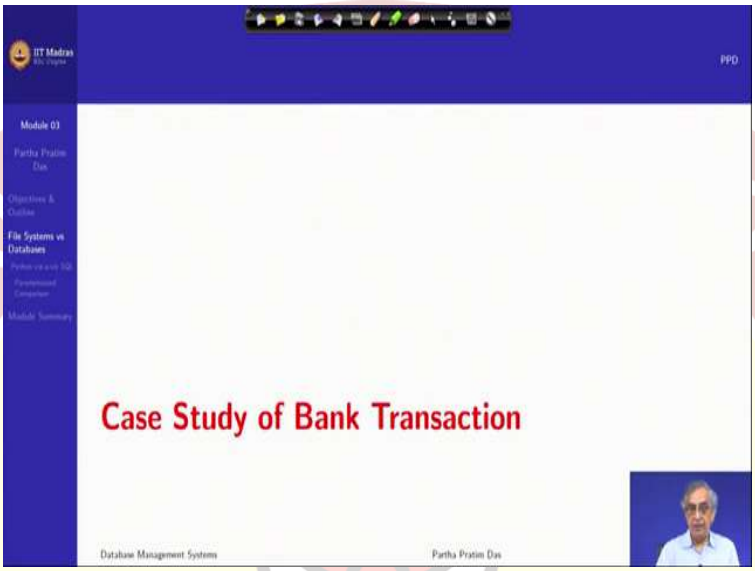


So, we will continue on that, but look at the same question from a little different perspective that in terms of the electronic systems of data and records management, how does it compare between a file based data management and DBMS? This question is going to be addressed through a very rudimentary example.

So, what we will do is we will talk about a problem and very small problem and a very small transaction of payment. And we will present comparative solutions, one, using Python using a flat file like .csv file as I talked of and the other is a database system using SQL language. And then we will try to compare between these two solutions. And as we introduced several parameters in

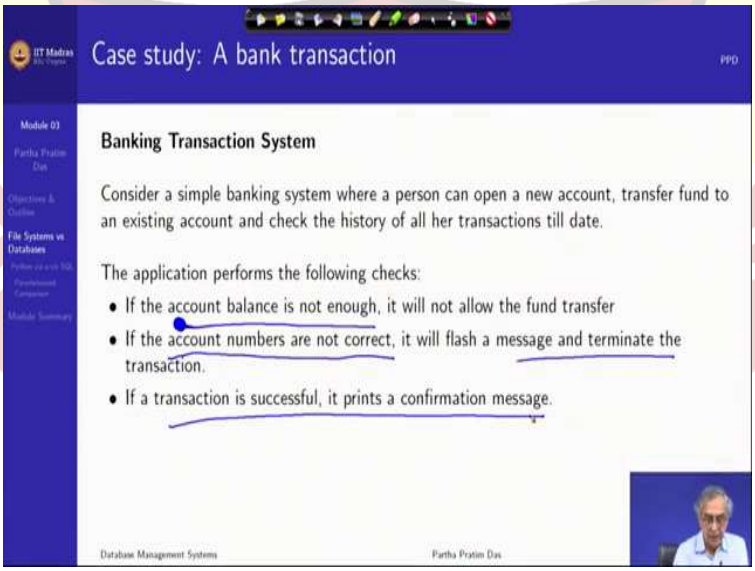
the last module of durability, scalability, security and so on, we will point by point try to highlight between the two solutions, which one has strength in which area and may not be so good in other areas.

(Refer Slide Time: 2:45)



So, it is a case study of a simple bank transaction.

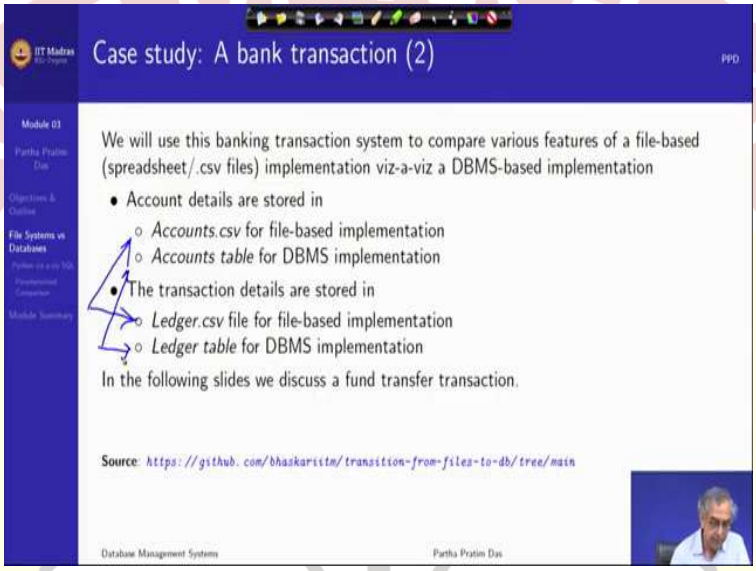
(Refer Slide Time: 2:51)



So, consider a simple banking system where a person can open an account, transfer funds to an existing account and check the history of her transactions till date, something all of us are doing most of the days. So, this application is defined with three basic conditions.

One is if the account balance, if the account balance is not enough, then it will not allow the fund transfers. If the account numbers are not correct, it will also give you a message and terminate the transaction. And if a transaction is successful, it will print a confirmation message, the simplest of transaction that you can have, but at the same time very representative of a major chunk of transactions that we do every moment.

(Refer Slide Time: 4:01)



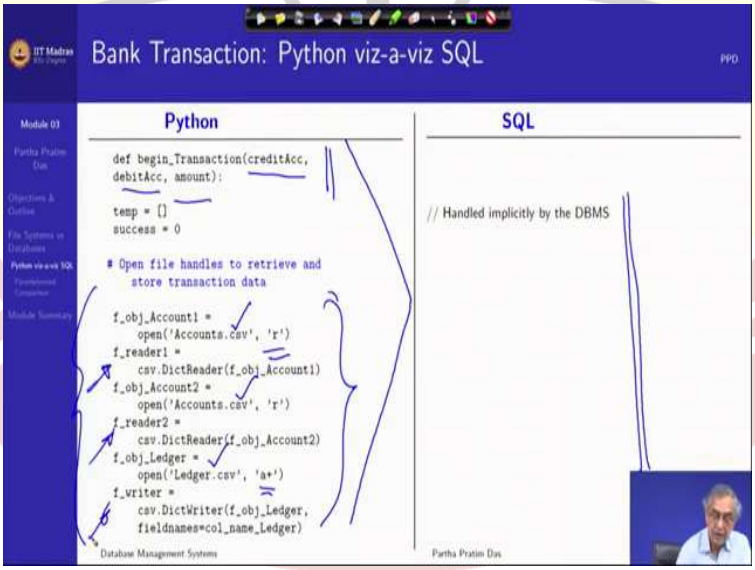
So, what is bank transaction system as we said, we will compare between Python based or file based system using Python as a programming language and a database application. So, what all we have here is a, for the file based system, we assume that there are two, there are sorry, I should really reset, for the file based system we have two different CSV ledger. One is called accounts.csv, there is called ledger.csv.

Account.csv has the information about the accounts and transactions are stored in the ledger.csv. Similarly, for a database application we use two tables, you will get to know what tables are in general in our DBMS and so on, but you can just conceptually think of them as tables right now. So, it is an accounts table and it is ledger table which correspond to it.

(Refer Slide Time: 5:23)

And with that, let us get into a comparison Python viz-a-viz SQL SQL incidentally is short form of structured query language. Now, SQL itself has become the name, which is a way to query a database system and get answers and make the transactions happen.

(Refer Slide Time: 5:43)



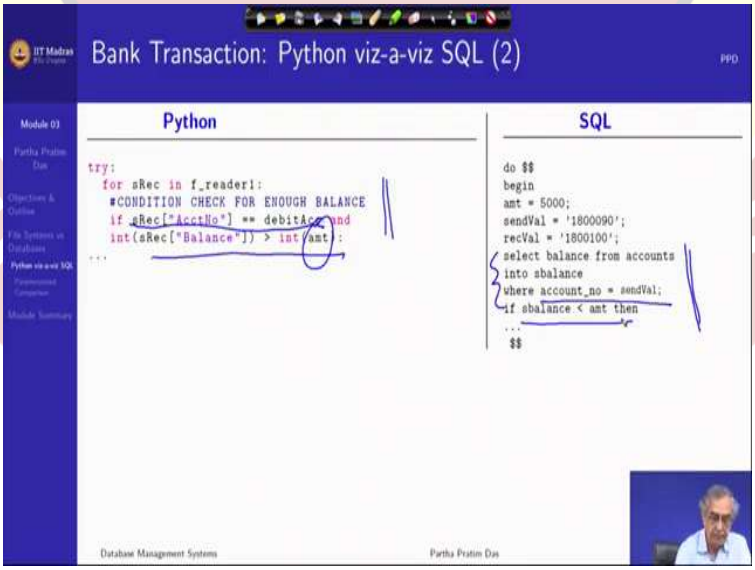
So, on left I show you the Python code. So, let me highlight a few things. So, this is I am defining a Python function as you can understand begin transaction. So, you need to know three components, credit account, debit account and the amount that needs to be transacted. So, with that, what all you need to do is this whole stuff is kind of an initialize, handle initialization that

you have to open the CSV file, accounts.csv file in the read mode, and that is your reader 1, you have to open it again in the read mode that is your reader 2, why do you have to open it twice, because you have to deal with two different accounts.

And then you have a ledger, which here, which you have to, which you are opening a+ that is append mode, so, that if you write to it, the transaction will be written here. So, this is. Now, on a SQL site, I have not given the corresponding code, but they will be done by the database as well. The only interesting part of the difference here is, which I would like to highlight to you is that, that for the Python function, you are doing it inside the function. So, every time you do the transaction, you will have to do all this, all this code for every function transaction function you write, you will have to do this.

For a database, that is not the case. For a database, you, database has certain mechanisms through which once you start the database, all these remain initialized, and you can just start using it. So, not every transaction need to do this kind of initial setup of open file, creating handles and all of that. So, that is first point when it makes a lot of difference in terms of using SQL things are a lot more simple in that way, but well, this is more Python naturally is till date is more understandable to you. So, let us move on, let us see what is the next.

(Refer Slide Time: 8:12)

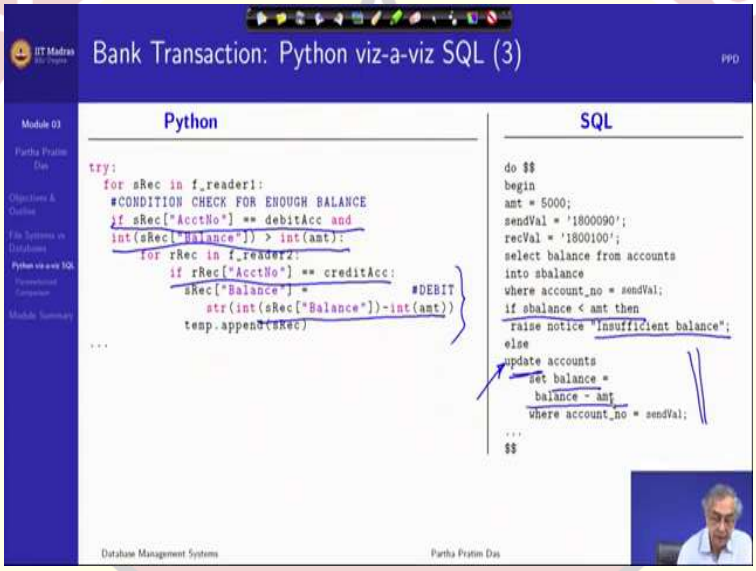


So, this is the transaction actually next happening, as you can find out so there is a condition check for enough balance, you are checking that account number, if the account number matches the

debit account, then you want to see that the balance in the balance field you have more amount than what is the amount what you will have to debit here.

Here, you do that same thing this select from balance into sbalance, where account number is the send account is the equivalent of the task you are doing in Python in terms of SQL and you check if it is less than that amount, then it will not be allowed. I mean condition check has been done in positive negative two different ways. So, the first task is done that you have the check that the account balance has been enough, let us move on.

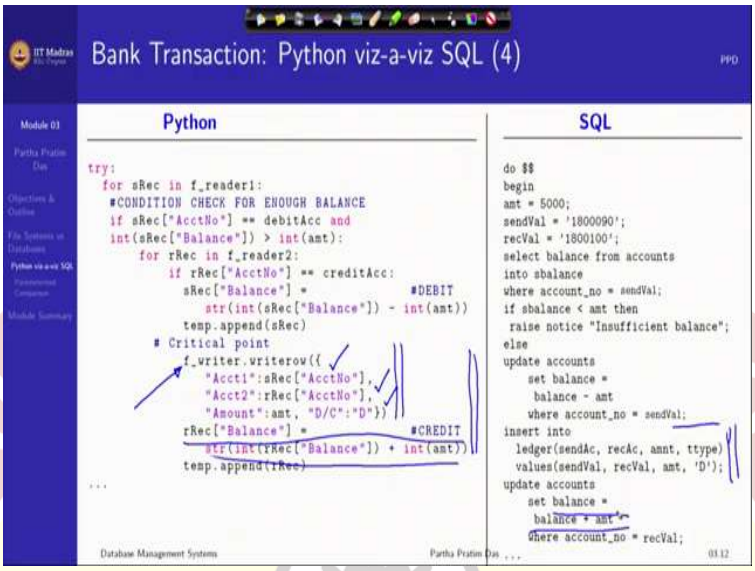
(Refer Slide Time: 9:32)



So, we have set up the handles we have checked account balance. The next is we will have to debit. So, this is the in the credit account you check, you, if, if this, these ifs are important. We will see what is their consequence, these lines have become dirty, if this is the credit account then you proceed by doing this basic debit that you change debit balance from balance to balance minus amount.

You do the same thing if this is less than amount, it is insufficient balance. So, we go to else, where it is valid to do the debit and you do what is known as an update, that is you are setting balance to balance minus amount, SQL way of doing it. So, we have done three things, we have opened the handles, we have checked the amount in the debit account, and we have done the basic debit operation.

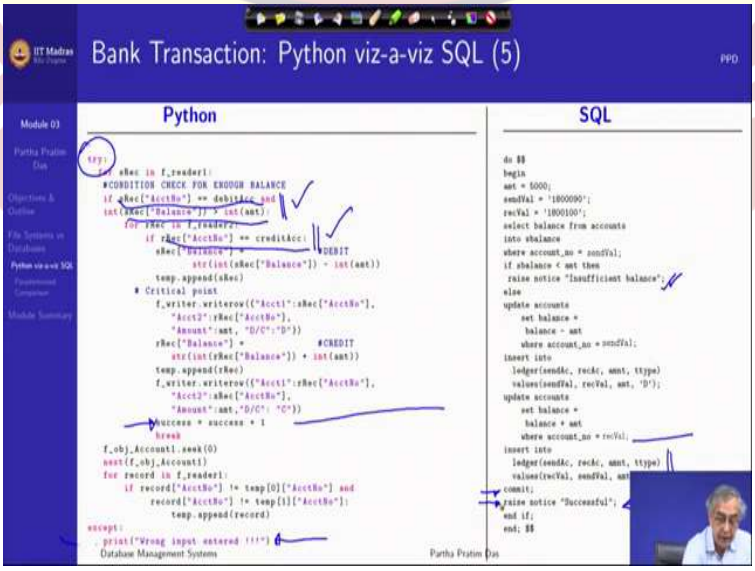
(Refer Slide Time: 11:07)



Then the next is where you do the credit that is you are finally writing to the ledger about the transaction so that the credit account gets balanced plus this amount. So, the credit is actually happening and you are writing down the transaction here this is your debit account, this is your credit account, this is a form of what transaction has happened.

Looking here, we had done up to this now you have insert ledger, so in the ledger you had making entry of this debit credit transaction and you're again equivalently increasing the credit account balance. So, four things have now happened.

(Refer Slide Time: 12:05)

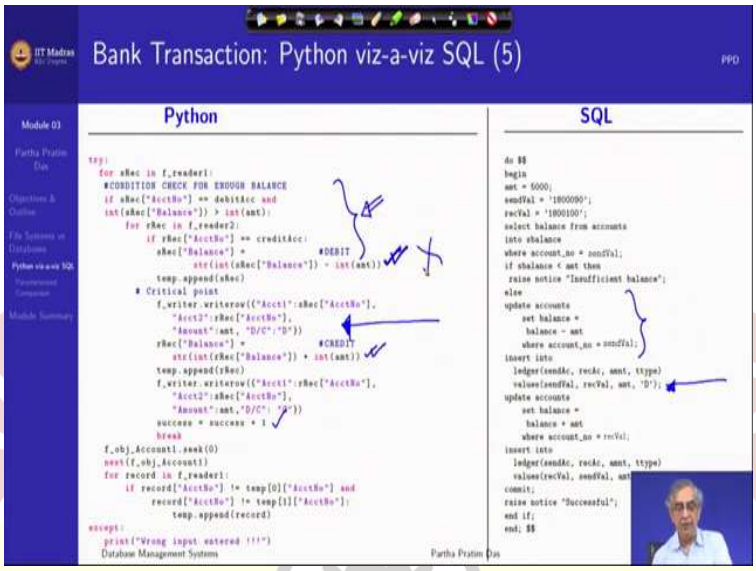


So, let us look at the entire thing together now I have put in the final end. So, this much we had seen before. If this has happened, then you have success and this much we had seen before. So, you insert into the send ledger as well. And then the important thing is in SQL, you do what is called the commit, commitment so far whatever has been happening were kind of temporary, and when you commit, it actually goes into the database, goes into, it is done, nothing is going to change it.

Similarly, here, you do not have an equivalent in that way, but there are few things you need to know is if you check this for this condition here, as we check for this condition here, what happens if the condition is invalid. So, we talked about the fact that this is being checked on debit account, this is being checked on amount, being checked on credit account, what if the debit account is not a valid account or the amount is not enough. So, that is where the stripe comes into play.

That is we are doing this within what is known as an exception block, try catch kind of block. So, if any one of these conditions fail, then the in, then the control jumps the rest and comes directly here. And from this you can get to, get the message that it is wrong input was entered. Whereas in case of SQL, if you had insufficient balance, you will get it here and if you are unable, if you have been able to commit, then you will raise a notice give a message that it is successful. So, this is kind of the side by side of how things can be done using either the file system or the database system.

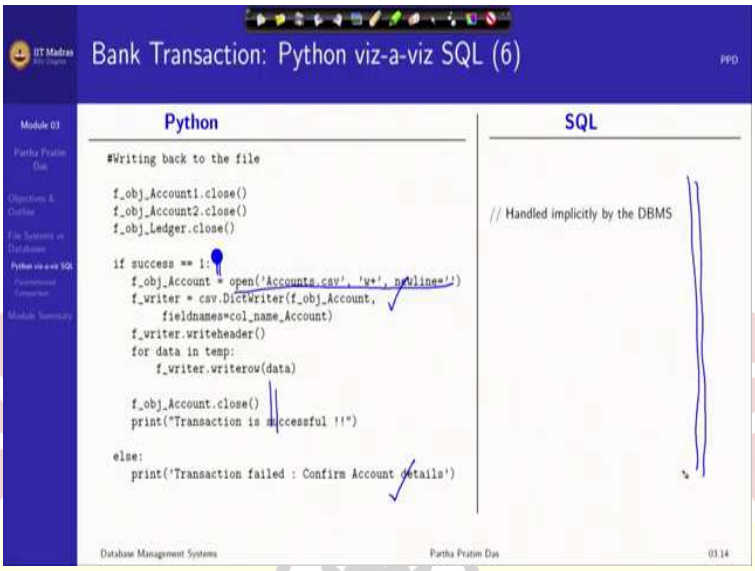
(Refer Slide Time: 14:29)



The critical point is suppose so all the checks here are between these. So, if this part has gone through then in any case, you have a success. Now, what happens, so what has been done by now? You have already debited the account, debit account by the amount. Now, suppose this is the point say where your system fails, so it could not complete. So, your CSV file is left in an inconsistent state, because it has the debit, this debit has happened. But you have not been able to complete the information of the transaction or actually credit it. So, the change is incomplete.

In a similar situation happening in SQL, say what is the equivalent point here is equivalent point here is at this point. Now, if a failure happens at this point, then the database has a built in mechanism to what we call rollback. That is, it can undo the effect of this debit that was done, which is not possible in this case, unless you write a whole lot of additional code, and even then there are problems. So, a very, I am just trying to show you a very simple issue here of what is between managing a transaction with the flat file using Python. And the advantage of using SQL.

(Refer Slide Time: 16:28)

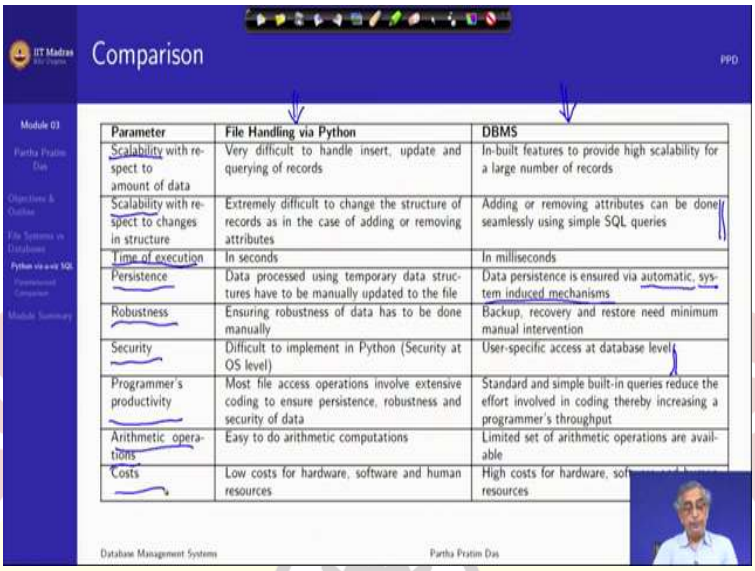


Just to close this is the final part that you will have to do. Again, you have all these on the left hand side, all these, write backs, because you check, just to see what you are checking, you are checking, I am sorry, you are checking if success is 1, success is 1 means that your transaction have happened successfully in this Python code.

Then what you will have to do, you will have to again open the accounts file and write back this whole thing. So, you write this back, you do all this file close and all that stuff. And otherwise, if it is not success, you say confirm the count details. So, either the debit account or all the credit account was not properly given.

Well, in SQL, much of this does not need to be done, because when you have done commit, all this has happened, there are no handles opened for this particular transaction. So, there is nothing that you need to specifically close or take care of when this transaction is completed. Or it has failed either way, the database will automatically take care of it.

(Refer Slide Time: 18:03)



Parameter	File Handling via Python	DBMS
Scalability with respect to amount of data	Very difficult to handle insert, update and querying of records	In-built features to provide high scalability for a large number of records
Scalability with respect to changes in structure	Extremely difficult to change the structure of records as in the case of adding or removing attributes	Adding or removing attributes can be done seamlessly using simple SQL queries
Time of execution	In seconds	In milliseconds
Persistence	Data processed using temporary data structures have to be manually updated to the file	Data persistence is ensured via automatic, system induced mechanisms
Robustness	Ensuring robustness of data has to be done manually	Backup, recovery and restore need minimum manual intervention
Security	Difficult to implement in Python (Security at OS level)	User-specific access at database level
Programmer's productivity	Most file access operations involve extensive coding to ensure persistence, robustness and security of data	Standard and simple built-in queries reduce the effort involved in coding thereby increasing a programmer's throughput
Arithmetic operations	Easy to do arithmetic computations	Limited set of arithmetic operations are available
Costs	Low costs for hardware, software and human resources	High costs for hardware, software and human resources

So, you can see that, well, it is it is possible to actually do equivalent operations between a database system using SQL. And by using Python, or some other language, we have decided to stick to Python, because that is becoming very, very popular in terms of data handling, in small scale, even in some large scales. We will talk about that.

But when you do it with Python, there is lot more of coding, lot more of vulnerability that you will have to bear with. So, it is not that, it is not possible to develop data management applications using file systems, but it is far more robust, reliable, efficient, productive, cost effective to develop those applications, particularly those which need to be scalable using a standard database management system.

You might question that if that is the case, then why does the option for file based data management is still provided? Why does Python provide these options for doing database kind of applications in Python? The reason is simple. There are always requirements for doing small things. There are always requirements of doing a lot of requirements of doing things by the side where having a complete database system has other consequences of price, the overhead of data, best installation, configuration, expertise and all that which may not be just justified here.

You know that you are making an data management application for hundred, 200 records and you have simple transactions and you really do not care that much about security, you do not need any

astronomical speed, you do not really mind if at times you lose the data, you can always bring it back from your backups and so on. So, a Python based, file based system will be equally workable.

So, here what I have tried to do and in the subsequent slides, I have tried to elaborate on them is a comparison between these two things, one is file based handling and the database based handling. So, one is doing with the scalability, it is very difficult to scale to handle insert, delete and update query of records in a file based system in database, it is just built in.

Scalability with respect to changes in structure, because for example, today, if I have a certain system and tomorrow, if the rules of the business get changed, for example, think of every financial year, every budget changes the rules for how the income tax will be computed. So, the way you prepare your tax computation tables, what are your different paths of income that are considered which has wavered, which has, what has rebate and so on, so forth, keeps on changing almost on a yearly basis.

Now, given that, it will be very difficult to adjust the programming codes for it, whereas for a database system removing or adding attributes as simple as a couple of lines of SQL code. Time of execution of course is significantly different in Python, it could be seconds depending on of course, the size, if the size is big, it will get into minutes or maybe, may not even go through. In database it is milliseconds or much less, typically it is in the order of microseconds.

Persistence is a big question that how durable the data is, how easily will it stay. Now, in, while you use file handling, certainly, you are having to do the basic computations or transaction deals through data structures. So, data structures are all ways of maintaining the data in memory. So, after that is done you have to update the file as you saw, you had to write back accounts.csv.

So, if you have only a part of the task done without the right back properly happening, then it is very easy to lose data. Whereas in database persistence is ensured through automatic and system induced mechanisms. They are already built in, these are the things we will learn how to do it, we are now trying to just understand what is there and why it is there.

Robustness is a big difference, getting the robustness that is if something goes wrong, if some transactions go wrong, some data gets corrupted, some file gets bad, then how do you get back the original, this is not easy at all in a file system based structure. Whereas in a database, you have standard backup recovery and restore mechanism by which you will be able to do this very easily.

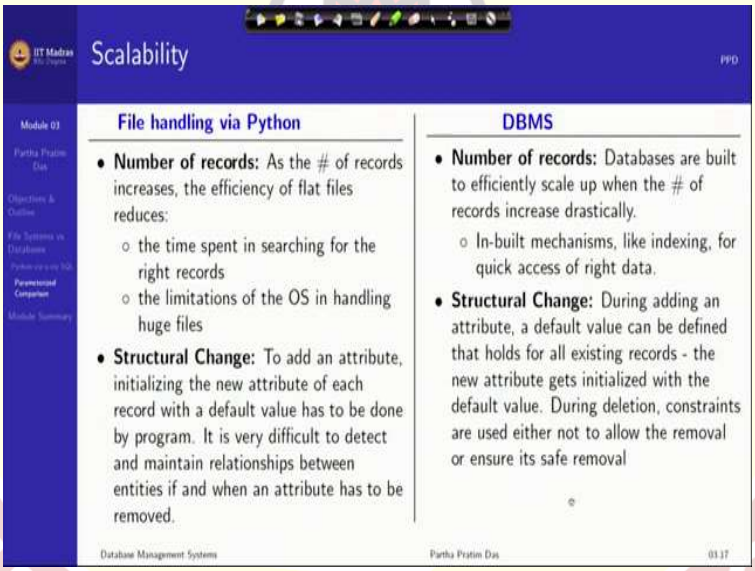
Security I have talked about several times in a file, you may have a security at an operating system level. That is I mean, can I log into this system, those kind of which are very easy to break relatively. Whereas in database you can have multiple layers, multiple levels of security, which can be user specific. That is what security I have and what security my database administrator who controls the whole thing as are different, maybe the database administrator can look into everything.

The, it could be that the systems engineer has a level of security by which she can check every table, but not the actual data. That is, interesting, you can check the table, you can check if the table is consistent and all that, but you cannot look inside the data because you do not have the authority, these kinds of multiple levels of authentication, authority and security are possible in databases, which are not possible file handling system.

This is a big issue, particularly more and more today, programmer's productivity, that is how easy it is for the programmer to produce the output given the problem that it she has at hand. If the programmer needs more effort to generate the solution or to maintain the solution, the enterprise overall incurs more cost. So, you want to make systems which are easier to manage, which are easier to manipulate, easier to maintain. And that certainly is more so, lot more for a database system than file based coding system where you will have to do a whole lot of coding overall.

Arithmetic operations are easy in file handling via Python kind of system, you can do a lot more of arithmetic computations, whereas in a database system, you have options, but the options are limited, often you will have to make use of some other high level language for that purpose. And finally, the cost is significantly low for your Python based handling and in database the costs are typically higher, but so, at a certain scale, the databases do make good sense, but in a lower scale, the file handling by Python is what is the way to go.

(Refer Slide Time: 27:18)



Time and Efficiency

PPD

Module 03

Parameterized Comparison

File handling via Python	DBMS
• The effort needed to implement a file handler is quite less in Python • In order to process a 1GB file, a program in Python would typically take few seconds.	• The effort to install and configure a DB in a DB server is expensive & time consuming • In order to process a 1GB file, an SQL query would typically take few milliseconds.

- If the number of records is very small, the overhead in installing and configuring a database will be much more than the time advantage obtained from executing the queries.
- However, if the number of records is really large, then the time required in the initialization process of a database will be negligible as compared to the t using SQL queries.

Database Management Systems Partha Pratim Das

Persistence, Robustness, Security

PPD

Module 03

Parameterized Comparison

File handling via Python	DBMS
• **Persistence**: Data processed using in-memory data structures stay in the memory during processing. After updates, these are manually updated to the file on disk • **Robustness**: Ensuring consistency, reliability and sanity is manual via multiple checks. On a system crash, a transaction may cause inconsistency or loss of data. • **Security**: Extremely difficult to implement granular security in file systems. Authentication is at the OS level.	• **Persistence**: Data persistence is ensured via automatic, system mechanisms. The programmer does not have to worry about the data getting lost due to manual errors • **Robustness**: Backup, recovery & restore need minimum manual intervention. The backup and recovery plan can be devised for automatic recovery on a crash • **Security**: DBMS provides user-specific access at the database level with restriction for to view only access

Database Management Systems Partha Pratim Das 03.19

INDIAN INSTITUTE OF TECHNOLOGY MADRAS
सिद्धिर्भवति कर्मजा

Programmer's Productivity

PPD

Module 03

Parthu Pratim Das

Objectives & Outline

File Systems vs Databases

Python vs vs SQL

Parameterized Comparison

Module Summary

File handling via Python

- **Building the file handler**: Since the constraints within and across entities have to be enforced manually, the effort involved in building a file handling application is huge
- **Maintenance**: To maintain the consistency of data, one must regularly check for sanity of data and the relationships between entities during inserts, updates and deletes
- **Handling huge data**: As the data grows beyond the capacity of the file handler, more efforts are needed

DBMS

- **Configuring the database**: The installation and configuration of a database is specialized job of a DBA. A programmer, on the other hand, is saved the trouble
- **Maintenance**: DBMS has in-built mechanisms to ensure consistency and sanity of data being inserted, updated or deleted. The programmer does not need to do such checks
- **Handling huge data**: DBMS can handle even terabytes of data - Programmer does not have to worry

Database Management Systems Partha Pratim Das 03.20

Arithmetic Operations

PPD

Module 03

Parthu Pratim Das

Objectives & Outline

File Systems vs Databases

Python vs vs SQL

Parameterized Comparison

Module Summary

File handling via Python

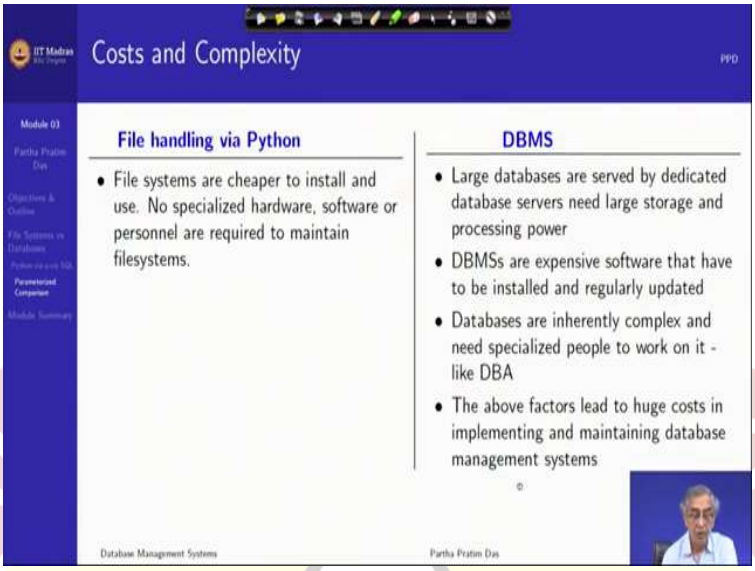
- **Extensive support for arithmetic and logical operations**: Extensive arithmetic and logical operations can be performed on data using Python. These include complex numerical calculations and recursive computations.

DBMS

- **Limited support for arithmetic and logical operations**: SQL provides limited arithmetic and logical operations. Any other complex computation has to be done outside the SQL.

Database Management Systems Partha Pratim Das

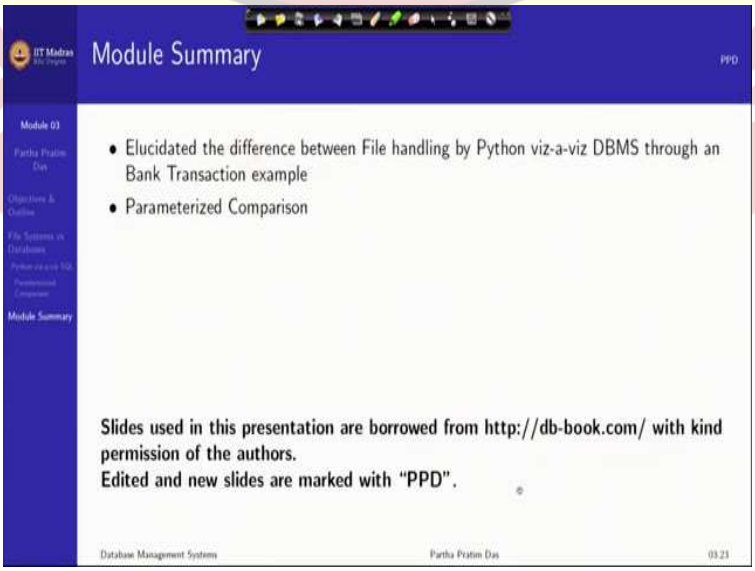
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
सिद्धिर्भवति कर्मजा



So, this is a broad comparison of the two approaches, I will not go into the specific parameterized comparison part because this is this I have kept so that you can study each one of the parameters and the comparison in depth and get prepared. So, it talks about comparison in terms of scalability, comparison in terms of time and efficiency, comparison in terms of persistence, robustness and security.

Comparison in terms of programmer's productivity, these are just the points that I said those are more detailed here. This is more for your records and information for preparing better. The comparison in terms of arithmetic operations and comparison in terms of cost and complexity.

(Refer Slide Time: 28:16)



So, with this, in this module, I have talked about the difference between file handling by Python, viz-a-viz database management system through a simple bank transaction example. And we have provided a couple of slides of details with this difference between file handling by Python and database system for your own self study. With this, I will close on Module 3. Thank you very much for your attention. We meet again for module 4.

INDIAN INSTITUTE OF TECHNOLOGY MADRAS
सिद्धिर्भवति कर्मजा